Deadlock-Free Event-Based Coordination of a Mobile Robot Fleet in a Grid Warehouse

Jakub Wilczyński, Nylan Santos, Antoni Piałucha, Maksymilian Tulewicz

Instructor: Gurjinder Singh October 2026

---

1 The Problem of the Project

Subject. The project concerns the supervisory, event-based control of a fleet of 10 autonomous mobile robots (AGVs) operating in a small automated warehouse. The warehouse floor is modelled as a grid of cells. Robots travel along one-way aisles between **5 pick-up stations** located at the storage racks, **3 delivery (outbound) stations**, **4 charging ports** and a **parking area**. Customer orders arrive over time; each order requires a robot to collect a load at a given pick-up station and bring it to a given delivery station (Fig. 1). Every grid cell is treated as a shared resource with unit capacity: a robot must acquire the next cell before it may enter it and releases the previous cell after leaving it.

Why it is important. Fleets of mobile robots are the backbone of modern intralogistics (e.g. goods-to-person systems in e-commerce warehouses). When many robots share narrow aisles and intersections, purely local obstacle avoidance is not sufficient: robots can end up in a *circular wait*, in which each robot waits for a cell held by the next one, and the whole system stops (*deadlock*). Such situations are difficult to recover from automatically and directly reduce throughput. Moreover, robots have limited battery capacity, so transport tasks compete with charging for robot time and for a small number of charging ports. These are typical discrete-event phenomena: the relevant dynamics is the order of events (a robot enters a cell, a load is picked, an order arrives, a battery is charged), not the continuous motion itself. The problem is therefore a natural application of the methods of event-based control: finite-state / resource-allocation models, deadlock avoidance and event prioritisation.

Expected result.

1. A formal discrete-event model of the warehouse (state space, events, enabling function EF(s) and transition function $next_state(s, e)$).
2. A supervisor that admits only safe events: collision freedom, deadlock freedom (algorithm $safe(s, e)$ with a correctness argument) and no robot running out of energy away from a charger.
3. A set of prioritisation rules (order assignment, conflicts at cells and intersections, charging policy) evaluated with respect to throughput and order lead time.
4. A Python discrete-event simulator used for fast experiments and for an exhaustive reachability check on a reduced instance.
5. A ROS 2 / Gazebo demonstration with 10 robots in the warehouse world, controlled by the same supervisor, plus a final report and a recorded demo.

2 Model and Control Concept

2.1 Warehouse layout and assumptions

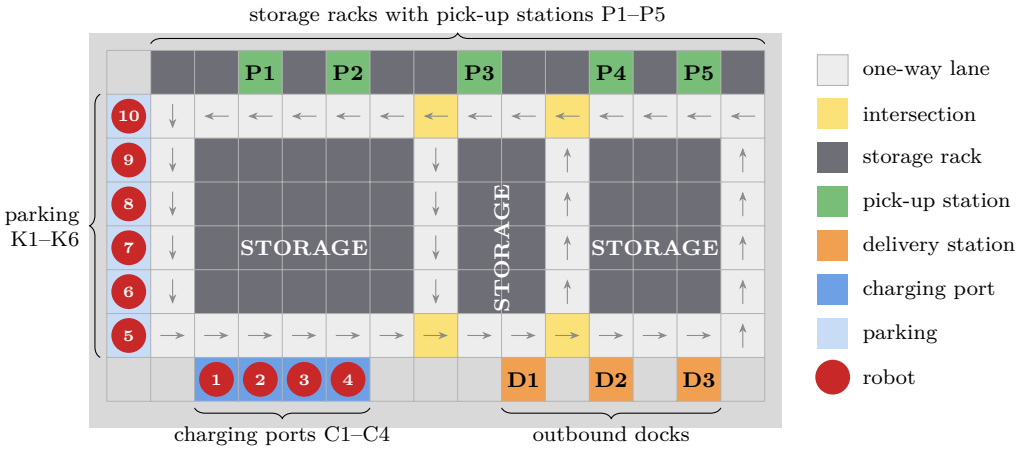


Figure 1: Proposed warehouse layout (15×8 cells). Lanes form a one-way loop with two cross connectors, creating four intersections. Stations, charging ports and parking places are dead-end *buffer cells*. The robots are shown in the initial state s_0.

Modelling assumptions:

- The floor is a set of cells V; a cell is $1 \times 1\,\mathrm{m}$ and can hold at most one robot. $V = L \cup \mathcal{B}$, where L are lane cells ($|L| = 44$) and $\mathcal{B} = P \cup D \cup C \cup K$ are buffer cells: pick-up stations $P = \{p_1, \ldots, p_5\}$, delivery stations $D = \{d_1, d_2, d_3\}$, charging ports $C = \{c_1, \ldots, c_4\}$ and parking places $K = \{k_1, \ldots, k_6\}$. The number of home cells $|C \cup K| = 10$ equals the fleet size, so every robot always has a place to go when it is idle.
- Admissible moves are given by a directed graph $G = (V, A)$ that encodes the lane directions of Fig. 1. Every buffer cell is a dead end connected to exactly one lane cell (in and out). For every pair of cells, $\mathit{route}(u, g)$ is a fixed shortest path in G and $\delta(u, g)$ is its length (number of moves).
- Robots $R = \{1, \ldots, 10\}$ carry at most one load. Loading, unloading and charging take a finite, not exactly known time.
- An order $o = (p_o, d_o)$, $p_o \in P$, $d_o \in D$, is generated by the warehouse management system at unknown times. The battery level is measured in moves: one move costs one unit, full capacity is B (e.g. $B = 400$), and b_{res} is a safety reserve.

2.2 System state

The state of robot $i \in R$ is the tuple

$$s_i = (H_i,\ m_i,\ b_i,\ o_i,\ g_i),$$

where

- H_i – the sequence of cells held by the robot: $H_i = (u)$ when the robot stands in cell u, $H_i = (u, v)$ while it moves from u to v (it holds both cells). We write $\mathrm{pos}_i = u$ for the first element;
- $m_i \in M$ – the operating mode (Fig. 2), where $M = \{\textsc{Idle}, \textsc{ToPick}, \textsc{Loading}, \textsc{Loaded}, \textsc{ToDrop}, \textsc{Unloading}, \textsc{ToCharge}, \textsc{Charging}, \textsc{ToPark}\}$;
- $b_i \in \{0, \ldots, B\}$ – battery level;
- $o_i \in \mathcal{O} \cup \{\perp\}$ – the order assigned to the robot;
- $g_i \in \mathcal{B} \cup \{\perp\}$ – the goal buffer cell, reserved for robot i until it arrives there.

The symbol $\perp$ means "none": $o_i = \perp$ – the robot has no order, $g_i = \perp$ – the robot has no reserved goal (it is not driving anywhere). The system state is $s = (s_1, \ldots, s_{10}, Q, n)$, where Q is the FIFO queue of unassigned orders ($|Q| \le Q_{\max}$) and n is the number of delivered orders. The state space S is the set of all such tuples that satisfy the structural constraints (at most one robot per cell, at most one reservation per buffer cell). Derived quantities used below: $\mathrm{occ}(v) = i$ if $v \in H_i$ and $\perp$ otherwise; $\mathit{free}(g) \iff \mathrm{occ}(g) = \perp \wedge \neg\exists j\colon g_j = g$; next_i is the second cell of $\mathit{route}(\mathrm{pos}_i, g_i)$ and rest_i is the set of cells of this route not yet held by robot i.

Even with the routes fixed, $|S|$ grows roughly as $|V|^{10}$ ($|V| = 62$), i.e. well above 10^{17}, so the full model is not analysed by enumeration. Correctness is enforced on-line by the supervisor (Sec. 2.6) and checked exhaustively only on a reduced instance.

Initial state s_0. Robots 1–4 stand on charging ports $c_1, \ldots, c_4$ and robots 5–10 on parking places $k_1, \ldots, k_6$; for all i: $m_i = \textsc{Idle}$, $b_i = B$, $o_i = \perp$, $g_i = \perp$; $Q = \emptyset$ (continuous operation) or $Q = Q_0$, a given batch of N orders (batch scenario); $n = 0$.

Final state s_{fin} (batch scenario only). $Q = \emptyset$, $n = N$, and every robot is Idle on a charging port or a parking place. In continuous operation no final state is defined; liveness is used instead (Sec. 2.6).

2.3 Events

All events are parameterised by robot, order or cell. Controllable events are commands issued by the supervisor; uncontrollable events are reported by the robots or by the warehouse management system.

Event	Type	Meaning
$\mathsf{order}(o)$	uc	a new order $o = (p_o, d_o)$ arrives from the WMS
$\mathsf{assign}(i, o)$	c	order o is assigned to robot i
$\mathsf{dispatch}(i, g)$	c	robot i is given goal buffer cell g, which is reserved for it
$\mathsf{enter}(i, v)$	c	robot i is allowed to start moving into the neighbouring cell v
$\mathsf{arrive}(i)$	uc	robot i has completed the move and released the previous cell
$\mathsf{load}(i)$, $\mathsf{unload}(i)$	c	start of loading at p_{o_i} / unloading at d_{o_i}
$\mathsf{loaded}(i)$, $\mathsf{unloaded}(i)$	uc	end of loading / unloading
$\mathsf{charge}(i)$	c	start of charging
$\mathsf{charged}(i)$	uc	battery full, charging finished

Table 1: Events (c – controllable, uc – uncontrollable).

Hence (with all parameter values)

$$E_{\mathrm{con}} = \{\mathsf{assign}, \mathsf{dispatch}, \mathsf{enter}, \mathsf{load}, \mathsf{unload}, \mathsf{charge}\},$$
$$E_{\mathrm{uc}} = \{\mathsf{order}, \mathsf{arrive}, \mathsf{loaded}, \mathsf{unloaded}, \mathsf{charged}\}.$$

In the basic model all events are observable, $E_{\mathrm{obs}} = E = E_{\mathrm{con}} \cup E_{\mathrm{uc}}$: robots report the completion of each move and each operation, and the WMS reports new orders. The continuous pose of a robot between two cells is not observed by the supervisor; it is abstracted away by the pair $\mathsf{enter}/\mathsf{arrive}$. As an optional extension we may add the observable but uncontrollable event $\mathsf{fault}(i)$ (robot stopped by an obstacle or failure), which turns the cells held by robot i into temporarily blocked resources.

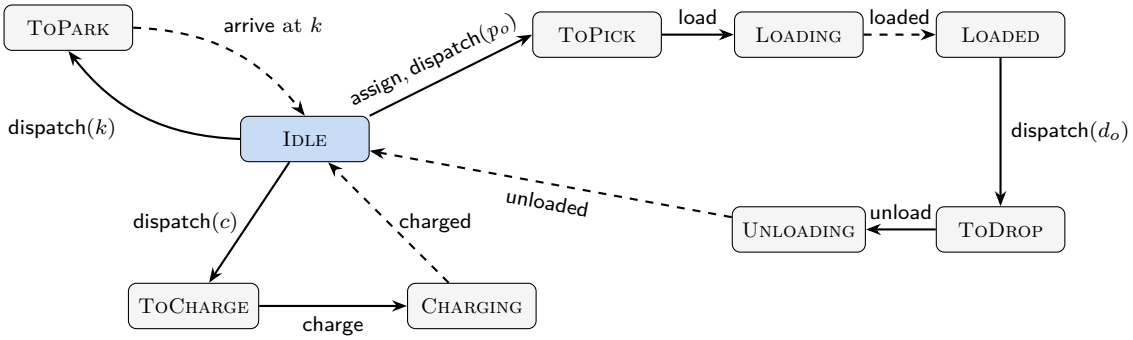


Figure 2: Mode automaton of a single robot (solid – controllable, dashed – uncontrollable). In the modes ToPick, ToDrop, ToCharge, ToPark the robot repeats enter/arrive along its route; this is where robots interact through the shared cells.

2.4 Events possible in a state

EF(s) is the set of events that can occur in state s. Each event has a *guard*, i.e. an enabling condition on the state; for example, enter(i, v) requires that cell v is free. An event e belongs to EF(s) if and only if its guard in Table 2 (column "Guard") is true in s. The guards are local and cheap to evaluate, so EF(s) is computed by a simple loop over robots, orders and buffer cells. Let $M_{\text{mov}} = \{\text{ToPick}, \text{ToDrop}, \text{ToCharge}, \text{ToPark}\}$ and let the energy-feasibility test be

$$\mathit{feas}(i, o) \iff b_i \geq \delta(\text{pos}_i, p_o) + \delta(p_o, d_o) + \max_{c \in C} \delta(d_o, c) + b_{\text{res}}.$$

Two observations: (i) the guard of enter(i, v) already excludes collisions, since v must be unoccupied, and buffer cells can be entered only by the robot that reserved them; (ii) EF(s) only describes what is *physically and logically possible*. The supervisor further restricts the controllable events with $\mathit{safe}(s, e)$ (Sec. 2.6) and chooses among the remaining ones with the priority rules (Sec. 2.7).

2.5 State transition

The transition function $s' = \mathit{next_state}(s, e)$, defined for $e \in \text{EF}(s)$, is given in the column "Effect" of Table 2. Variables not mentioned remain unchanged.

Event e	Guard: $e \in \text{EF}(s)$ iff	Effect: $s' = \mathit{next_state}(s, e)$
order(o)	$\lvert Q \rvert < Q_{\max}$	$Q \leftarrow Q \cdot o$ (append)
assign(i, o)	$o \in Q$, $m_i = \text{Idle}$, $o_i = \bot$, $\lvert H_i \rvert = 1$, $\mathit{feas}(i, o)$	$o_i \leftarrow o$, $Q \leftarrow Q \setminus \{o\}$
dispatch(i, g)	$\lvert H_i \rvert = 1$, $\text{pos}_i \in \mathcal{B}$, $\mathit{free}(g)$ and one of: (a) $m_i = \text{Idle}$, $o_i \neq \bot$, $g = p_{o_i}$; (b) $m_i = \text{Loaded}$, $g = d_{o_i}$; (c) $m_i = \text{Idle}$, $o_i = \bot$, $g \in C$; (d) $m_i = \text{Idle}$, $o_i = \bot$, $g \in K$	$g_i \leftarrow g$; $m_i \leftarrow$ (a) ToPick, (b) ToDrop, (c) ToCharge, (d) ToPark
enter(i, v)	$m_i \in M_{\text{mov}}$, $g_i \neq \bot$, $\lvert H_i \rvert = 1$, $v = \text{next}_i$, $\text{occ}(v) = \bot$, $b_i \geq 1$	$H_i \leftarrow (\text{pos}_i, v)$
arrive(i)	$H_i = (u, v)$	$H_i \leftarrow (v)$, $b_i \leftarrow b_i - 1$; if $v = g_i$: $g_i \leftarrow \bot$ and, if $m_i = \text{ToPark}$, $m_i \leftarrow \text{Idle}$
load(i)	$m_i = \text{ToPick}$, $H_i = (p_{o_i})$	$m_i \leftarrow \text{Loading}$
loaded(i)	$m_i = \text{Loading}$	$m_i \leftarrow \text{Loaded}$
unload(i)	$m_i = \text{ToDrop}$, $H_i = (d_{o_i})$	$m_i \leftarrow \text{Unloading}$
unloaded(i)	$m_i = \text{Unloading}$	$m_i \leftarrow \text{Idle}$, $o_i \leftarrow \bot$, $n \leftarrow n + 1$
charge(i)	$m_i = \text{ToCharge}$, $H_i = (c)$, $c \in C$	$m_i \leftarrow \text{Charging}$
charged(i)	$m_i = \text{Charging}$	$m_i \leftarrow \text{Idle}$, $b_i \leftarrow B$

Table 2: Enabling function EF(s) and transition function $\mathit{next_state}(s, e)$.

2.6 Correctness and safety

The behaviour of the system is considered correct if the following properties hold in every state reachable under the supervisor.

(S1) Collision freedom: $H_i \cap H_j = \emptyset$ for all $i \neq j$.

(S2) Energy safety: a robot never runs out of energy away from a charging port, i.e. $b_i \geq \delta(\mathrm{pos}_i, C)$ at all times.

(S3) Deadlock freedom: there is no set of moving robots in which each robot waits for a cell held by another robot of the set (circular wait).

(L) Liveness / reachability: every order in Q is eventually delivered; in the batch scenario s_{fin} is reachable from every reachable state (the supervised system is non-blocking).

(S1) and (S2) follow from the guards by induction over events: s_0 satisfies both, enter only acquires an unoccupied cell, buffer cells are reserved exclusively, arrive only releases cells, and $\mathit{feas}(i, o)$ guarantees enough energy to finish the order and reach any charger. (S3) does not follow from the guards: robots on the one-way loop can fill a cycle of cells and block each other. It is enforced by a *deadlock-avoidance* test in the spirit of the Banker's algorithm for resource allocation systems, applied to each controllable event before it is issued.

Algorithm 1 $\mathit{safe}(s, e)$ – deadlock-avoidance test

```
1: if e is not of the form enter(i, v) then return true      ▷ other events do not acquire lane cells
2: s' ← next_state(s, e)
3: A ← { j ∈ R : H_j ⊈ B }                                   ▷ robots currently holding a lane cell
4: F ← V \ ⋃_{j∈A} H_j                                       ▷ cells not held by active robots
5: repeat
6:     find j ∈ A such that rest_j ⊆ F                       ▷ j can reach its reserved goal
7:     if such j exists then
8:         A ← A \ {j};  F ← F ∪ H_j                        ▷ j parks in its buffer cell and frees its cells
9: until no such j exists
10: return (A = ∅)
```

Proof idea. Call a state *safe* if Algorithm 1 (the loop part) empties $\mathcal{A}$, i.e. there is an order in which all active robots can drive to their reserved goal cells one after another. (1) s_0 is safe, because all robots are in buffer cells. (2) The supervisor issues a controllable event only if $\mathit{safe}(s, e)$ holds, so the next state is safe. (3) Uncontrollable events cannot destroy safety: arrive only releases a cell and shortens rest_i, so the old completion order is still valid; the other uncontrollable events do not change any H_j or rest_j. Hence all reachable states are safe. In a safe state the first robot of the completion order can always move, since its next cell is free, so a circular wait cannot occur (S3). A robot that reaches its goal stands in a dead-end buffer cell that lies on no other route, so it never blocks other robots. Together with fairness (every started motion or operation ends in finite time) and the aging rule of Sec. 2.7, which prevents starvation of robots waiting to leave a buffer cell, this gives (L). The test runs in $O(|R|^2\,\ell)$ time, where ℓ is the maximum route length, which is negligible for 10 robots. The test is conservative (sufficient but not necessary); a less restrictive variant that claims cells only up to the next intersection will be compared experimentally.

Demonstration of correctness.

1. The invariant-based proof above, written in full in the report.
2. **Exhaustive reachability analysis** on a reduced instance (e.g. 3–4 robots on a smaller version of the layout) with a BFS over the state graph generated by EF and *next_state*. Without the supervisor, the analysis is expected to find reachable deadlock states (shown as counterexample

traces). With $safe(s, e)$, no deadlock or collision state must be reachable, and s_{fin} must be reachable from every state.

3. **Runtime monitors** in the 10-robot simulations that check (S1)–(S3) after every event and log every violation (target: zero violations over long randomised runs).

2.7 Event prioritisation and efficiency

Efficiency criterion. The primary criterion is the **throughput** $\Theta = n/T$ (orders delivered per hour of operation). Secondary criteria are the mean and maximum **order lead time** (time from $\mathsf{order}(o)$ to $\mathsf{unloaded}$ for o), the total time robots spend blocked waiting for cells, and fleet utilisation.

Conflicts. Typical conflicting events in one state are: two robots requesting the same cell, i.e. $\mathsf{enter}(i, v)$ and $\mathsf{enter}(j, v)$, one order that could be assigned to several robots, several robots competing for one charging port, and several robots that want to leave buffer cells into the same lane. In each state the supervisor first computes the candidate set $\{e \in E_{\text{con}} \cap \mathrm{EF}(s) : safe(s, e)\}$ and then applies the rules below.

R1. Cell conflicts. Priority classes, from highest: (1) robot with critical battery ($b_i < b_{\text{crit}}$) heading to a charger; (2) loaded robot (ToDrop); (3) robot already on the lane before a robot leaving a buffer cell ("main road first"); (4) ToPick; (5) ToPark and non-critical ToCharge. Within a class the robot with the longest waiting time wins (*aging*, which prevents starvation); remaining ties are broken by robot index.

R2. Intersections. A robot may enter an intersection cell only if it can also take the cell behind the intersection ("do not block the box"). This is a cheap pre-filter for the *safe* test.

R3. Order assignment. Orders are served in FIFO order, and overdue orders are moved forward. Each order is assigned to the idle robot with the smallest $\delta(\text{pos}_i, p_o)$ that satisfies $feas(i, o)$. When several orders and robots are free at the same time, a minimum-cost assignment (Hungarian algorithm) can be used instead.

R4. Charging policy. If $b_i < b_{\text{low}}$, charging takes priority over new orders for robot i. Idle robots with $b_i < b_{\text{opp}}$ charge opportunistically when a port is free; otherwise idle robots go to parking.

R5. Station clearing. An idle robot standing in a pick-up or delivery station is dispatched away before robots in parking places are moved, so that stations are free for the next orders.

Use for efficiency. The rules reduce the time loaded robots spend waiting and keep stations and intersections free, which shortens order lead times. They also spread charging over idle periods, so fewer robots are unavailable during peaks. The benefit will be quantified by comparing the proposed policy with a baseline (first-come-first-served cell access, random idle robot, charging only when the battery is low) at several order arrival rates, using the same random order sequences for both policies.

2.8 Proposed modelling approach

The system is modelled as a **Resource Allocation System (RAS)** expressed as a **finite-state automaton**: grid cells are reusable resources of unit capacity, robots are processes that acquire and release these resources in the sequence given by their routes, and the state is the vector described above. This choice is appropriate because: (i) deadlock in AGV systems is exactly a circular-wait problem of a RAS, so established deadlock-avoidance theory (Banker-type safety tests) applies directly; (ii) the model is finite, so EF and *next_state* can be executed directly as code, and the reduced instance can be checked by enumeration; (iii) the same model can be translated into an equivalent Petri net (cells as places with one token, robot moves as transitions), which we will use for presentation and, if time permits, for structural analysis (siphons) with an existing Petri-net tool.

2.9 Planned implementation

- **Model core** (Python 3): grid and route definitions, EF, *next_state*, *safe* and the priority rules, as one library with unit tests (pytest).
- **Discrete-event simulator** (Python): random order streams and random operation times, KPI logging, the reachability analysis of the reduced instance, and policy comparison (plots with matplotlib).
- **ROS 2 (Humble/Jazzy) + Gazebo**: a warehouse world generated from the same grid description, 10 lightweight differential-drive robots, a robot-level node that executes cell-to-cell motion and publishes `arrive`, `loaded`, ... events, and a `fleet_supervisor` node that runs the model core and issues controllable events. Visualisation of states and reservations in RViz. Lightweight robot models keep the simulation real-time on standard laptops without a dedicated GPU.
- **Tools**: Git/GitHub, Overleaf (LaTeX) for reports, recorded demo video.

3 Work Plan and Timetable

The project is planned for 10 weeks. Week 1 starts on 19 October 2026 and the work is completed before the winter break. Exact milestone dates will be aligned with the course schedule.

	Task	Responsible	Weeks
T1	Literature (RAS, AGV deadlock avoidance), requirements, layout	all (lead: A)	1
T2	Formal model: state, events, EF, *next_state*	A, B	1–3
T3	Safety: $safe(s, e)$, invariant proof	B	2–4
T4	Python model core, DES simulator, reachability analysis	B, C	3–5
T5	Gazebo warehouse world and robot models	C	2–5
T6	ROS 2 robot-level nodes (motion, event reporting, battery)	D	3–6
T7	`fleet_supervisor` node, integration of 10 robots	A, D	6–8
T8	Priority policies, experiments, KPI evaluation	C, A	7–9
T9	Reports, demo video, final presentation	all (lead: A)	1, 5, 9–10

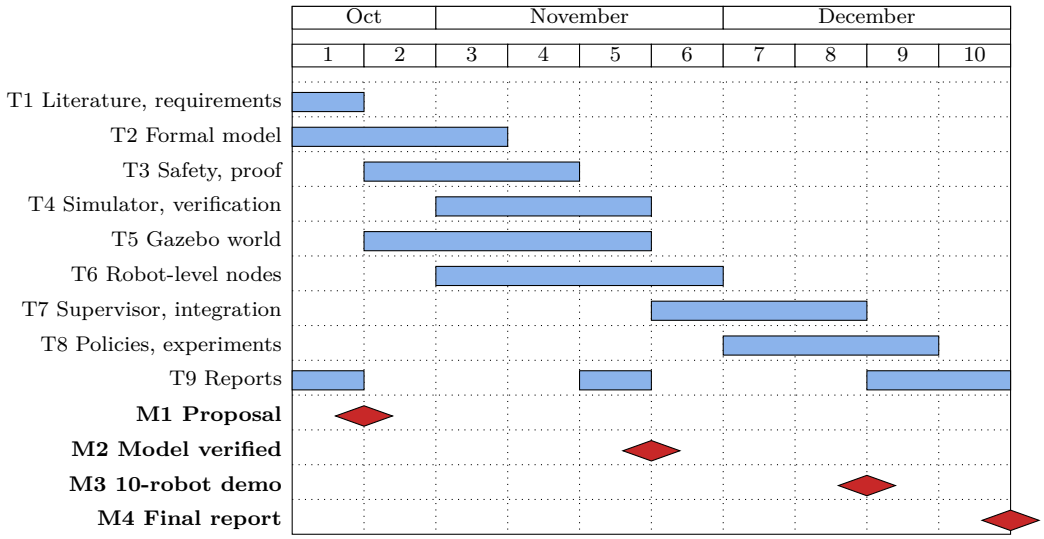


Figure 3: Gantt chart (weeks of the semester).

Milestones: **M1** (week 1, 20 Oct 2026) – proposal submitted; **M2** (week 5, 20 Nov 2026) – formal model, simulator and safety verification on the reduced instance; **M3** (week 8, 11 Dec 2026) – 10-robot ROS 2/Gazebo demonstration; **M4** (week 10, 22 Dec 2026) – final report and presentation.

4 Delivery

- **M1 – Proposal report** (this document): problem, model and control concept, plan. PDF and hard copy.
- **M2 – Interim report**: complete formal model, *safe* algorithm with proof, reachability results (including the deadlock counterexample without the supervisor), first simulator KPIs. Attachment: archive (Git tag `m2`) with the model core, simulator and tests.
- **M3 – Demonstration**: live or recorded demo of 10 robots in Gazebo with a short status note. Attachment: Git tag `m3`, ROS 2 packages, launch files, demo video.
- **M4 – Final report**: full description, policy comparison, conclusions, user guide. Attachments: final source archive (tag `v1.0`), simulation logs, plots and video. PDF.

Disclosure: reports are intended for the instructor and the team. The source code is kept in a private repository during the semester and may be published under the MIT licence after grading, with the consent of all members.

5 Project Management

- **Structure and coordination.** The coordinator (Jakub Wilczyński) assigns tasks, sets internal deadlines, merges changes into the main branch, and is the contact person for the instructor. Each task has one responsible member (Sec. 6).
- **Meetings and monitoring.** A weekly 30-minute online stand-up (Monday evening) reviews the status of each task on a GitHub Projects board (issues per task, *To do / In progress / Review / Done*). Milestone readiness is reviewed one week before each milestone. Consultations with the instructor take place at milestones or on request.
- **Documents and communication.** Code on GitHub, reports in a shared Overleaf project, other files in a shared Google Drive folder. Day-to-day communication on a team Discord server; messages are answered within 24 h on working days.
- **Software development rules.** Feature branches and pull requests; every pull request is reviewed by at least one other member and must pass automated tests (GitHub Actions, pytest, linter); PEP 8 and ROS 2 package conventions; issues are linked to commits.
- **Decisions and conflicts.** Technical decisions are made by consensus at the weekly meeting. If no consensus is reached, a majority vote decides; in a 2:2 tie the coordinator has the deciding vote. Unresolved organisational conflicts are escalated to the instructor.
- **Intellectual property.** The results are joint work of all four members with equal rights, subject to the regulations of Wrocław University of Science and Technology. Each member may use the results in their own portfolio or thesis with attribution to the team.

6 Team

	Name	Index	Responsibility
A	Jakub Wilczyński – **coordinator**	275459	coordination, formal model (T2), supervisor node and integration (T7), experiments (T8), reports (T9)
B	Nylan Santos	300482	formal model (T2), safety algorithm and proof (T3), model core and verification (T4)
C	Antoni Piałucha	275411	simulator (T4), Gazebo warehouse world (T5), policies and KPI evaluation (T8)
D	Maksymilian Tulewicz	275421	ROS 2 robot-level nodes (T6), supervisor integration (T7), demo video